

Nombres Completos:

Josseline Jhomayra Granda Chacha

Materia:

Herramientas Informáticas para el Despliegue de Diagramas

Ciclo/Carrera:

3ro Ciclo de Desarrollo de Software

Tema:

Base de datos con arrays

Fecha de entrega:

10/10/2026

Docente:

ING. Jhostin Vacacela

Ciudad:

Cuenca

1. Objetivo

Crear una API sencilla en NestJS para administrar usuarios usando un array como almacenamiento temporal. El proceso incluyó definir la estructura de los usuarios, conectar el array con el servicio, implementar las operaciones CRUD (incluido PUT para reemplazar un usuario), validar los datos recibidos con `ValidationPipe` (error 400), responder 404 con `NotFoundException` cuando el usuario no existe y probar todas las peticiones desde Postman.

2. Estructura utilizada

Se creó una clase llamada `Temporal` para definir los datos de cada usuario: identificador, nombre y edad. Después se declaró un array llamado `usuarios` con tres registros iniciales (Juan, María y Pedro).

Elemento	Función
Temporal	Define id, nombre y edad.
usuarios	Guarda los registros en memoria.
DTO (CreateUsuarioDto, UpdateUsuarioDto)	Define y valida los datos que llegan al crear, reemplazar o actualizar.
ValidationPipe	Valida automáticamente los datos de entrada y responde 400 si son incorrectos.
Service	Contiene la lógica para administrar el array y lanza `NotFoundException` (404) si el id no existe.
Controller	Recibe las peticiones HTTP.

3. Proceso realizado

1. **Definir los datos.** Se organizó la clase `Temporal` con los campos id, nombre y edad.
2. **Crear el array.** Se preparó el array `usuarios` con tres registros iniciales.
3. **Crear los DTO con validaciones.** Se instalaron `class-validator` y `class-transformer`. `CreateUsuarioDto` exige un nombre (texto no vacío) y una edad (entero mayor o igual a 0). `UpdateUsuarioDto` tiene los mismos campos como opcionales.
4. **Activar ValidationPipe.** Se configuró de forma global en `main.ts` con `whitelist`, `forbidNonWhitelisted` y `transform`.
5. **Conectar el servicio.** Se implementaron `create`, `findAll`, `findOne`, `replace` (PUT), `update` (PATCH) y `remove`. Si el id no existe, el servicio lanza `NotFoundException`.

6. **Implementar PUT.** Se agregó `@Put(':id')` en el controlador para reemplazar el usuario completo (nombre y edad obligatorios).
7. **Corregir los ids.** El id nuevo ya no se calcula con `usuarios.length + 1`, porque repetía ids al eliminar usuarios. Ahora se usa un contador que solo aumenta.
8. **Iniciar la aplicación.** La terminal mostró que NestJS inició correctamente y registró las rutas del controlador.
9. **Probar en Postman** cada operación, incluidos los casos de error 400 y 404.

4. Peticiones HTTP utilizadas

El prefijo de las rutas es `/usuario` y el puerto es el 3000.

Método	Ruta de ejemplo	Uso	Respuesta esperada
GET	http://localhost:3000/usuario	Consultar todos los usuarios.	200
GET	http://localhost:3000/usuario/2	Consultar el usuario con id 2.	200 (404 si no existe)
POST	http://localhost:3000/usuario	Crear un usuario con nombre y edad.	201 (400 si los datos son inválidos)
PUT	http://localhost:3000/usuario/2	Reemplazar completamente el usuario con id 2.	200 (400 si falta un campo, 404 si no existe)
PATCH	http://localhost:3000/usuario/2	Actualizar solo los campos enviados.	200 (400 si un dato es inválido, 404 si no existe)
DELETE	http://localhost:3000/usuario/2	Eliminar el usuario con id 2.	200 (404 si no existe)
HEAD	http://localhost:3000/usuario	Obtener solo las cabeceras (incluye `X-Recurso: usuario`), sin cuerpo.	200

Método	Ruta de ejemplo	Uso	Respuesta esperada
HEAD	http://localhost:3000/usuario/2	Comprobar si el usuario existe, sin cuerpo.	200 (404 si no existe)
OPTIONS	http://localhost:3000/usuario	Consultar los métodos permitidos (cabecera `Allow`).	204
OPTIONS	http://localhost:3000/usuario/2	Consultar los métodos permitidos sobre un usuario.	204

Diferencia entre PUT y PATCH: PUT reemplaza el usuario completo, por eso exige nombre y edad. PATCH cambia únicamente los campos que se envían.

5. Ejemplos para Postman

Crear un usuario (POST). Seleccionar POST, escribir `http://localhost:3000/usuario` y en Body elegir raw y JSON:

```
{
  "nombre": "Ana",
  "edad": 21
}
```

Reemplazar un usuario (PUT). Seleccionar PUT, usar `http://localhost:3000/usuario/2` y enviar todos los campos:

```
{
  "nombre": "Maria Jose",
  "edad": 23
}
```

Modificar un usuario (PATCH). Enviar solo el campo que se desea cambiar:

```
{
  "edad": 22
}
```

Eliminar un usuario (DELETE). Seleccionar DELETE y usar la ruta con el id. No hace falta enviar Body.

Casos de error (validación y 404):

Prueba	Resultado esperado
POST con `{}`	400 Bad Request, con mensajes de nombre

Prueba	Resultado esperado
	y edad
PUT a `/usuario/2` con `{"nombre":"Solo nombre"}`	400 Bad Request
PUT a `/usuario/9999` con nombre y edad	404 Not Found
GET a `/usuario/99`	404, «Usuario con ID 99 no encontrado»
DELETE a `/usuario/99`	404 Not Found

6. Conceptos importantes

- **Array temporal:** una lista que guarda datos mientras el servidor está funcionando.
- **CRUD:** crear, consultar, actualizar y eliminar registros.
- **DTO:** define qué datos se reciben en una petición.
- **ValidationPipe:** componente de NestJS que valida los datos con las reglas del DTO y responde 400 si no se cumplen.
- **NotFoundException:** excepción de NestJS que responde 404 cuando el recurso no existe.
- **PUT:** reemplaza un recurso completo.
- **PATCH:** modifica solo una parte del recurso.
- **HEAD:** solicita las cabeceras de la respuesta sin el contenido.
- **OPTIONS:** permite consultar los métodos admitidos por una ruta.
- **Postman:** permite enviar peticiones HTTP y revisar las respuestas.

7. Resultado y observaciones

La aplicación compiló y arrancó correctamente, y NestJS registró las rutas: POST, HEAD, OPTIONS y GET en `/usuario`; HEAD, OPTIONS, GET, PUT, PATCH y DELETE en `/usuario/:id`. Las pruebas automáticas (10 pruebas unitarias) pasaron.

Problemas encontrados y corregidos:

Problema	Antes	Después
Ids duplicados	Con `usuarios.length + 1`, al eliminar un usuario y crear otros, dos usuarios quedaban con el mismo id.	Un contador que solo aumenta; el id nunca se repite.
Sin validación	`POST {}` creaba un usuario sin nombre	`ValidationPipe` con DTO:

Problema	Antes	Después
	ni edad.	responde 400 con el detalle de cada campo.
Usuario inexistente	GET devolvía 200 con cuerpo vacío; DELETE devolvía 200 con un texto.	`NotFoundException`: responde 404 con un mensaje claro.
Falta de PUT	No existía forma de reemplazar un usuario completo.	Se agregó `PUT /usuario/:id`.

Limitación: el array no es una base de datos permanente. Los datos se guardan en la memoria del servidor y se pierden al reiniciarlo, y entonces se recuperan los tres registros iniciales definidos en el código.

8. Conclusión

Se implementó una API de usuarios en NestJS con un array como almacenamiento temporal. Se completó el CRUD con PUT y PATCH, se validaron los datos de entrada con `ValidationPipe` (400) y se respondió 404 con `NotFoundException` cuando el usuario no existe. Se probaron todas las operaciones en Postman, incluidos los casos de error. Este método es útil para practicar el funcionamiento de una API antes de conectarla a una base de datos real.

9. Declaración de uso de inteligencia artificial

Declaro que utilicé **Claude (Anthropic)**, un asistente de inteligencia artificial, como herramienta de apoyo en este trabajo. La usé para:

1. revisar mi informe y cada archivo de mi repositorio;
2. detectar errores en mi código (ids duplicados, falta de validación y respuestas 200 donde correspondía 404);
3. proponer correcciones, como el método PUT, la validación con `ValidationPipe` y las excepciones `NotFoundException`;
4. explicar conceptos (DTO, HEAD, OPTIONS, códigos de estado) y guiarme paso a paso en IntelliJ;
5. mejorar la redacción del informe.

Yo ejecuté el proyecto, probé las peticiones en Postman, revisé el código final y comprendo su funcionamiento. Me responsabilizo del contenido entregado.

Registro de consultas (prompts)

N.º	Consulta	Objetivo	Palabras clave	Cómo llevó a la siguiente consulta
1	¿Cómo agregar los métodos HEAD y OPTIONS a un controlador de NestJS y probarlos en Postman?	Soportar HEAD (solo cabeceras) y OPTIONS (cabecera Allow) en la ruta /usuario	@Head, @Options, @Header, @HttpCode, Allow, Postman	Con las rutas funcionando, necesitaba comprobar que el resto del proyecto estuviera bien
2	¿Cómo revisar un CRUD de NestJS con un array en memoria (controller, service y DTO) para detectar errores de lógica?	Detectar fallos en el repositorio Tarea_en_clase	CRUD, array, controller, service, DTO, usuarios.length + 1, ids duplicados	La revisión mostró ids duplicados, falta de validación y respuestas 200 en lugar de 404
3	¿Cómo implementar PUT para reemplazar un recurso completo en NestJS y en qué se diferencia de PATCH?	Agregar @Put(':id') y el método replace en el servicio	@Put, @Patch, reemplazo total, actualización parcial, replace	Faltaba que los datos recibidos en POST y PUT fueran validados
4	¿Cómo validar el cuerpo de un POST con ValidationPipe, class-validator y class-transformer para responder 400?	Rechazar datos incompletos o inválidos en los DTO	ValidationPipe, @IsString, @IsNotEmpty, @IsInt, @Min, whitelist, forbidNonWhitelisted, 400	Con la validación lista, faltaba responder 404 cuando el id no existe
5	¿Cómo responder 404 con NotFoundException cuando un id no existe en el array?	Lanzar la excepción en findOne, replace, update y remove	NotFoundException, ParseIntPipe, find, findIndex, splice, 404	Debía aplicar los cambios en mi propio proyecto

N.º	Consulta	Objetivo	Palabras clave	Cómo llevó a la siguiente consulta
6	¿Cómo clonar un repositorio de GitHub en IntelliJ, instalar dependencias y ejecutar npm run lint, build y test?	Trabajar y verificar el proyecto en mi computadora	IntelliJ, Get from VCS, npm install, lint, jest, PowerShell	Con el proyecto verificado, faltaba actualizar el informe
7	¿Cómo documentar el uso de IA en un informe con una declaración y una tabla de prompts?	Cumplir el componente de IA de la rúbrica	declaración de IA, tabla de prompts, objetivo, palabras clave	Última consulta de esta entrega: reentregar informe y repositorio

Repositorio: https://github.com/jjgranda-gif/Tarea_en_clase.git